

Speed Programme Baseline

Speed Programme — Baseline Artefacts (Phase 0)

Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active
Status summary	none
Issues	none
Issues summary	none
Fixed	none
Fixed summary	none
Continuation	docs/CONTINUATION.md

Table of contents

- [1. Purpose](#)
- [2. Canonical scenarios S1-S4](#)
- [3. Harness components](#)
- [4. P0-T04 anti-bluff proof — 3-run variance](#)
- [5. Reproducing the capture](#)
- [6. Capture-file naming](#)

1. Purpose

This directory holds the committed measurement-baseline artefacts for the HelixCode speed programme (R4 phased plan docs/research/speed/04-phased-implementation-plan.md, Phase 0). Phase 0 changes no production code — it builds the measurement harness and captures before-state numbers so every later phase's speedup claim is falsifiable (CONST-035 / Article XI §11.9).

Task **P0-T04** specifically delivers the scenario fixtures + the canonical scenario runner. This README records its anti-bluff proof.

2. Canonical scenarios S1-S4

Defined in the shared manifest `helix_code/tests/performance/scenarios/scenarios.json`:

ID	Name	What it measures
S1	cold-start	CLI process spawn to ready prompt (Phase 1 target)
S2	llm-dispatch	single LLM generate-request dispatch (real provider via <code>HELIX_SPEED_LLM_URL</code> , else SKIP-OK per CONST-050)
S3	repomap-build	repo-map build I/O envelope over the large-repo fixture (Phase 2 target)
S4	content-search	content search (grep) over the large-repo fixture (Phase 2 target)

3. Harness components

- `helix_code/tests/performance/scenarios/` — Go package: the deterministic large-repo fixture generator (`fixture.go`), the scenario manifest loader (`manifest.go`), and the scenario runner (`runner.go`). Deterministic = same seed produces a byte-identical tree (proven by `fixture_test.go`).
- `helix_code/tests/performance/scenarios/cmd/runner/` — Go CLI entry point: generates fixtures and runs scenarios with per-scenario variance reporting.
- `scripts/testing/run_speed_scenarios.sh` — the shell runner: builds the Go runner, generates the fixture, executes S1-S4 N times, and writes a timestamped capture file here.

4. P0-T04 anti-bluff proof — 3-run variance

The harness must be stable enough to detect a 1.3x change — i.e. its run-to-run noise must be far below the magnitude of the changes later phases will claim. A coefficient of variation (CV) well under 30% satisfies this.

Capture `speed-scenarios-20260520T130750Z.txt` (go1.26.2, 2000-file fixture, 3 runs per scenario):

ID	name	mean_ms	min_ms	max_ms	CV %	verdict
S1	cold-start	2084.106	2083.117	2085.227	0.05	stable
S2	llm-dispatch	—	—	—	—	SKIP-OK (no HELIX_SPEED_LLM_URL)
S3	repomap-build	18.897	15.669	22.225	17.35	stable
S4	content-search	18.454	16.967	21.306	13.39	stable

All measured scenarios have $CV < 18\%$ — comfortably below the 30% bound and far below the 30-percentage-point gap a 1.3x change would produce. The harness can therefore reliably discriminate a 1.3x improvement or regression.

The integration test `TestRunner_StableAcrossThreeRuns` enforces this same invariant in CI-runnable form (it fails if $CV \geq 35\%$).

Captured run:

S3: samples=[6.188996 5.76131 8.061205] ms CV=18.34%

S4: samples=[6.307011 6.143849 7.842832] ms CV=13.86%

5. Reproducing the capture

From the repo root:

```
scripts/testing/run_speed_scenarios.sh --runs 3 --files 2000 --json
```

Or directly via the Go runner (from helix_code/):

```
cd helix_code
go run ./tests/performance/scenarios/cmd/runner -gen-fixture /tmp/fx -files 2000
go run ./tests/performance/scenarios/cmd/runner -run -runs 3 -fixture /tmp/fx
```

The fixture is deterministic — the same `--seed` always produces the identical tree, so a capture is reproducible byte-for-byte.

6. Capture-file naming

Capture files are named `speed-scenarios-<UTC-timestamp>.txt` (and `.json` with `--json`). They are committed reference artefacts (not build derivatives — they record a specific point-in-time measurement) and are retained as the before-state for later phase comparisons.